UNIVERSITÉ DE PERPIGNAN VIA DOMITIA
Département Informatique

INLINE

Application desktop de gestion quotidienne

Projet de Génie Logiciel
Licence 3 Informatique - Semestre 6
Année universitaire 2025-2026

Étudiant : Ryan ENCELADE - 92201942
Enseignant « Génie Logiciel » : Benjamin ANTUNES
Date de rendu : 12 avril 2026

1 SOMMAIRE

2 Table des figures 2
3 Introduction 3
4 1. Présentation de l'application 4
4.1 1.1 La Visualisation 4
4.2 1.2 Les Intentions 5
4.3 1.3 Les Notes 5
4.4 1.4 Les Modules 6
4.5 1.5 La Timeline 7
5 2. Architecture logicielle 8
5.1 2.1 Organisation en couches 8
5.2 2.2 Règle de dépendance 8
5.3 2.3 Principes SOLID appliqués 9
6 3. Diagrammes UML 10
6.1 3.1 Diagramme de classes 10
6.2 3.2 Diagramme de séquence 11
7 4. Patrons de conception 14
7.1 4.1 Patron Observer (comportement) 14
7.2 4.2 Patron Factory (création) 15
8 5. Outils de génie logiciel 16
8.1 5.1 Git 16
8.2 5.2 Docker 16
9 6. Tests unitaires 18
10 7. Organisation du travail 19
10.1 7.1 Tableau des tâches 19
10.2 7.2 Difficultés rencontrées 20
10.2.1 Évolution du modèle conceptuel 20
10.2.2 Généralisation des objets interactifs 20
10.2.3 Flexibilité des intentions 20
10.2.4 Positions des objets interactifs 20
10.2.5 Observer et signaux Qt 21
10.2.6 Sérialisation des blocs de notes 21
10.2.7 Objets partagés entre plusieurs environnements 21
10.2.8 Construction du datetime pour les événements planifiés 21
11 Conclusion 22

2 TABLE DES FIGURES

Figure 1 — Interface principale de la page de Visualisation
Figure 2 — Gestion des intentions et timeline journalière
Figure 3 — Editeur de notes avec blocs de contenu
Figure 4 — Module Pomodoro avec historique des sessions
Figure 5 — Timeline
Figure 6 — Diagramme de classes UML complet du projet
Figure 7 — Diagramme de séquence : création d'une note
Figure 8 — Tableau des tâches du projet

3 INTRODUCTION

Ce rapport présente le projet INLINE, réalisé dans le cadre du cours de Génie Logiciel de Licence 3 Informatique à l'Université de Perpignan Via Domitia. L'objectif du projet était de concevoir, développer et documenter une application logicielle complète en respectant les bonnes pratiques de l'ingénierie logicielle.

INLINE est une application desktop développée en Python avec PySide6. L'idée de départ était simple : pouvoir modéliser sa vie quotidienne de manière structurée, en partant de l'espace physique pour arriver à l'action planifiée. L'application repose sur un modèle centré autour de l'intention, qui constitue l'unité de base du système.

Le projet couvre les points suivants : architecture logicielle structurée, patrons de conception, tests unitaires, conteneurisation Docker, gestion de version Git et documentation complète.

Je fais une parenthèse pour préciser que j’ai initialement vu trop grand avant de commencer le développement. J’ai dû retirer de nombreuses fonctionnalités envisagées et revoir mes attentes à la baisse pour ce projet qui me tenait à cœur.

4 1. PRESENTATION DE L'APPLICATION

INLINE est un environnement numérique conçu pour représenter, organiser et analyser l'activité quotidienne d'un individu à travers un modèle structuré. L'application repose sur l'idée que la vie quotidienne peut être modélisée comme un ensemble d'intentions projetées, exécutées et observées dans le temps.

Le système s'articule autour de quatre piliers complémentaires : la Visualisation, les Notes, les Modules et la Timeline. Chacun joue un rôle précis et distinct, tout en restant interconnecté avec les autres via le concept central d'intention.

4.1 1.1 LA VISUALISATION

La visualisation constitue la porte d'entrée du système. Elle représente les espaces de vie de l'utilisateur sous forme de plans en deux dimensions. Chaque espace est composé d'environnements personnalisables, eux-mêmes peuplés d'objets interactifs.

Ces objets jouent le rôle de points d'ancrage cognitifs. L'idée est de permettre à l'utilisateur de projeter une intention à partir d'un contexte spatial familier, ce qui correspond à un principe connu de la visualisation mentale. L'interaction est volontairement simple : on clique sur un objet, on déclare une intention, et on peut lui associer un créneau horaire pour qu'elle devienne un événement visible sur la timeline.

Figure 1 — Interface principale de la page de Visualisation

4.2 1.2 Les Intentions

L'intention est l'unité centrale du modèle. Elle représente une volonté déclarée par l'utilisateur, sans nécessairement avoir de détails techniques ou de contraintes horaires. Elle peut exister seule, être planifiée, activée ou désactivée (focus), liée à un objet, à une note ou à un module.

Une intention peut être marquée comme active, ce qu'on appelle le focus. Quand une intention est en focus, les événements qui lui sont associés apparaissent en surbrillance sur la timeline, et l'interface s'adapte pour mettre en avant ce qui est lié à cette intention.

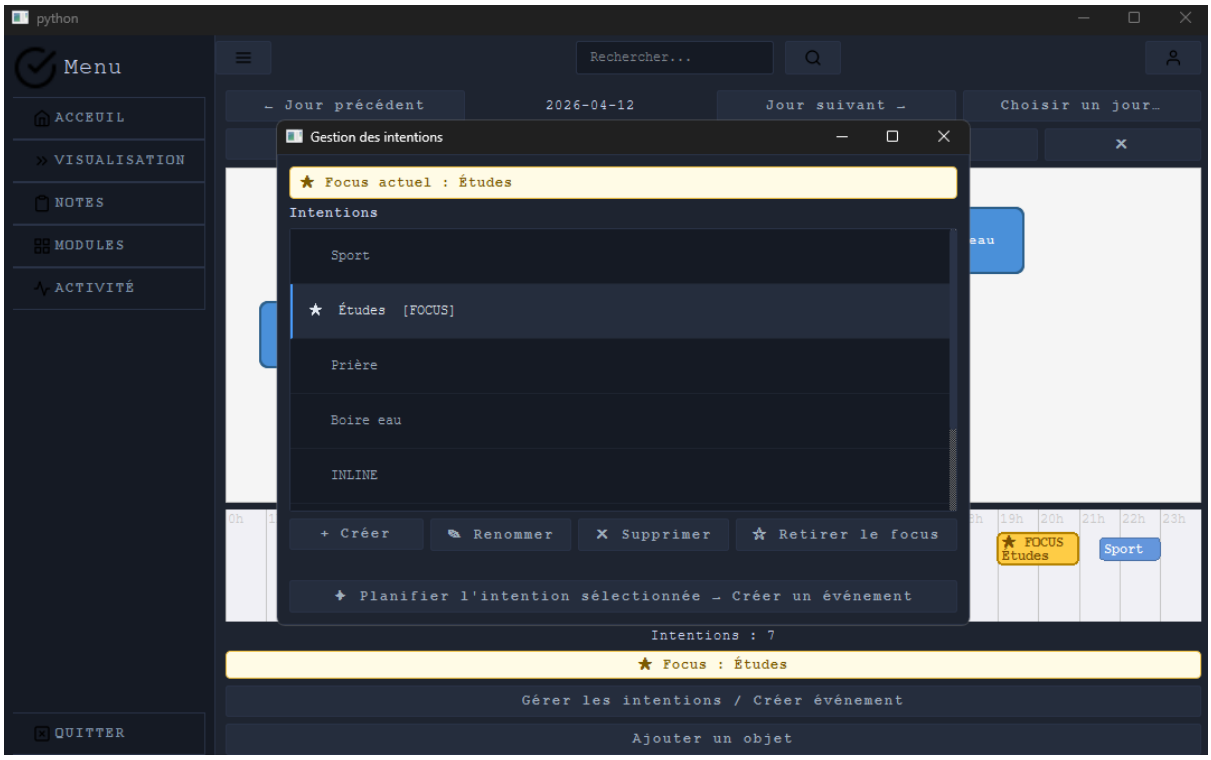

Figure 2 — Gestion des intentions et timeline journalière

4.3 1.3 Les Notes

L'espace de notes permet de structurer des idées, planifier ou conserver des informations liées à une intention ou complètement indépendantes. Les notes sont composées de blocs de contenu : titre avec niveau hiérarchique (H1, H2, H3), texte libre, checklist, ou tableau. Les blocs peuvent être ajoutés, supprimés et réordonnés librement.

Ce système de blocs s'inspire d'outils comme Notion. Il permet une grande flexibilité sans imposer une structure rigide. Une note peut être liée à une intention pour regrouper les informations associées à un objectif précis.

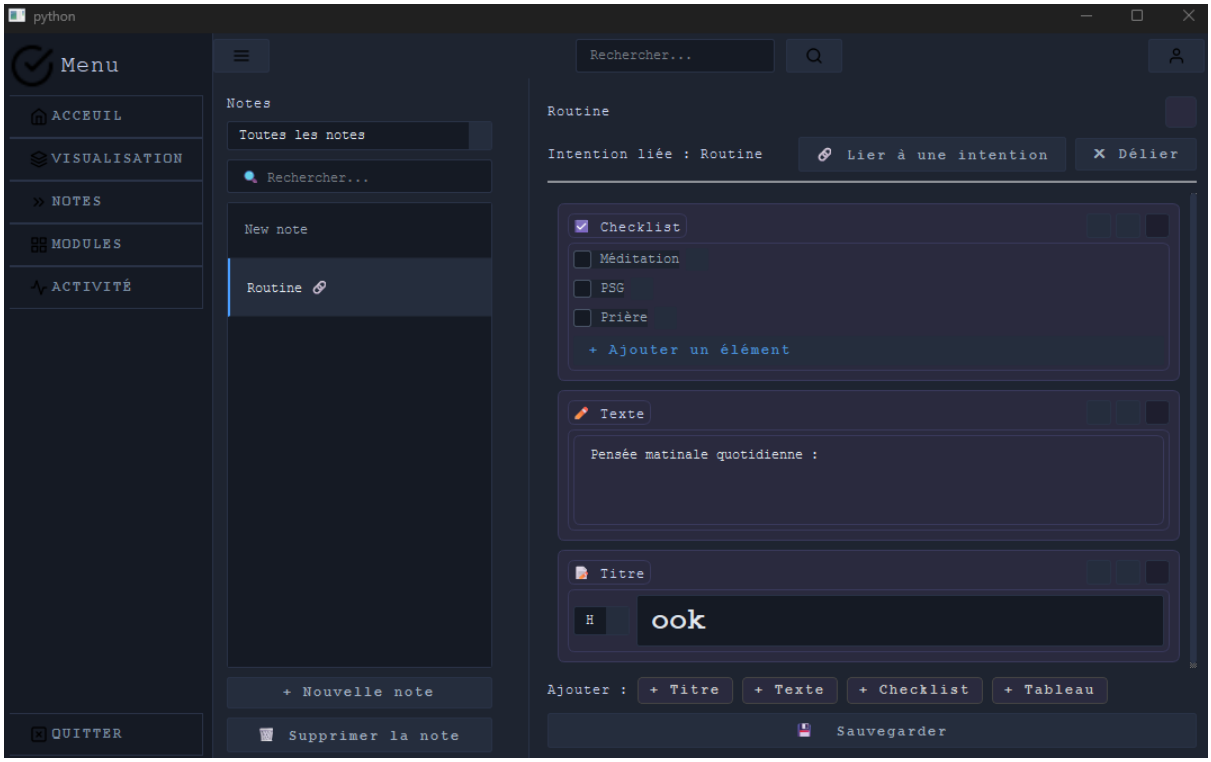

Figure 3 — Editeur de notes avec blocs de contenu

4.4 1.4 Les Modules

Les modules représentent la couche fonctionnelle spécialisée du système. Chaque module est une entité autonome dédiée à une responsabilité unique. Le seul module implémenté à ce stade est le Pomodoro.

Le module Pomodoro permet de créer des sessions de travail minutées avec des paramètres personnalisables : durée de travail, durée des pauses courtes, durée des pauses longues et nombre de sessions avant une longue pause. Les sessions sont persistées et un historique du jour est disponible. Un module peut être lié à une intention pour organiser les sessions autour d'un objectif précis.

L'architecture a été pensée pour permettre l'ajout de nouveaux modules sans toucher au noyau du système.

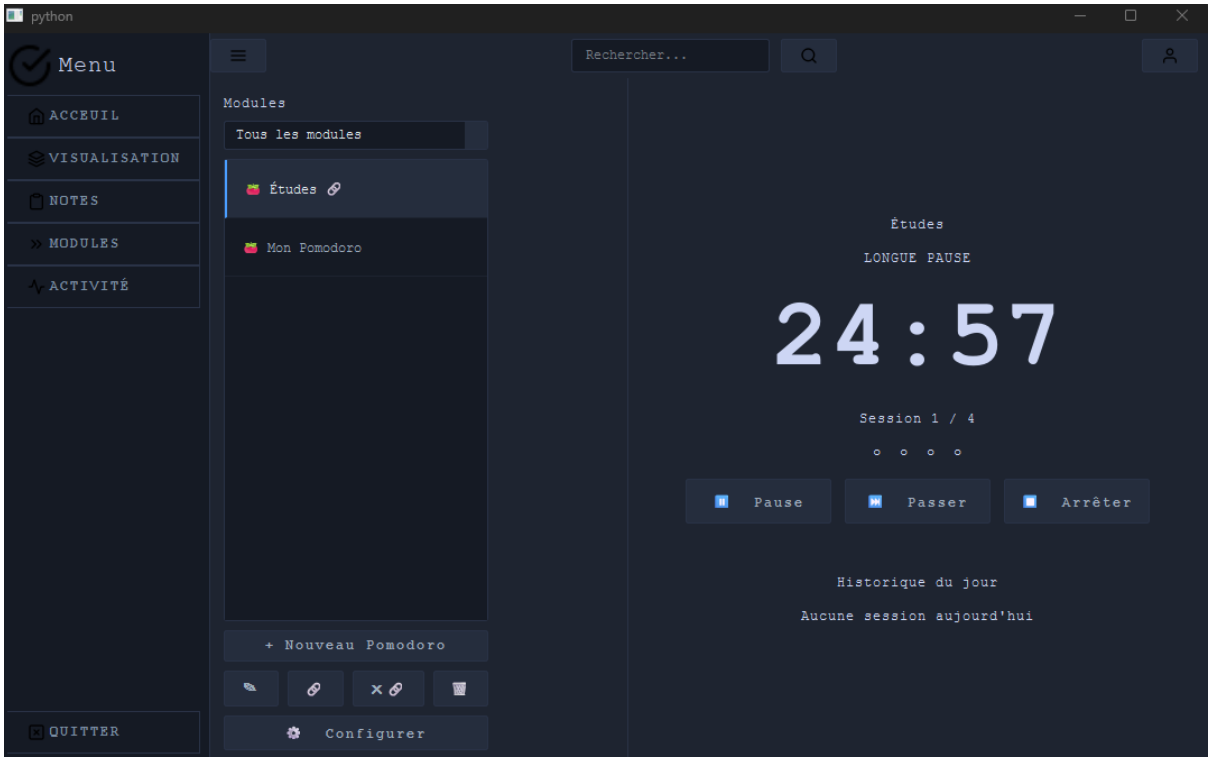

Figure 4 — Module Pomodoro avec historique des sessions

4.5 1.5 La Timeline

La timeline est la projection temporelle du système. Elle affiche les événements de la journée sous forme de blocs colorés sur une grille horaire. Chaque événement peut avoir trois statuts : planifié, complété ou annulé. En cliquant sur un événement, on peut le modifier, le reprogrammer, le compléter ou le supprimer.

La navigation entre les jours permet de consulter les journées passées ou de planifier des journées futures. La timeline ne stocke pas de données métier complexes ; elle expose uniquement les références nécessaires à l'affichage.

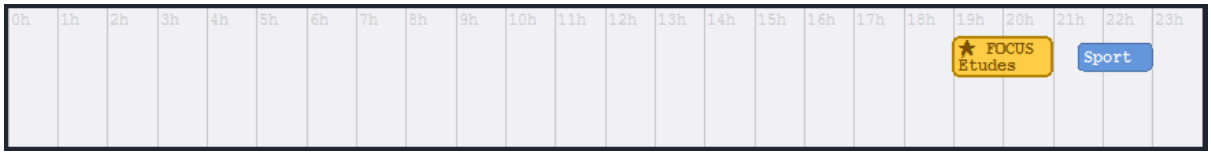

Figure 5 — Timeline

5 2. ARCHITECTURE LOGICIELLE

Pour ce projet, une architecture inspirée de la Clean Architecture a été choisie. Ce choix repose sur plusieurs raisons : le domaine métier est riche et complexe (intentions, événements, modules, visualisation), les règles métier doivent être indépendantes de l'interface graphique et du système de persistance, et la maintenabilité à long terme est une priorité car le projet amène naturellement à l'extensibilité.

L'architecture MVC et l'architecture en couches classique ont été écartées. MVC couple trop fortement le domaine et l'interface dans les projets de taille moyenne. L'architecture en couches simple ne garantit pas l'indépendance du domaine vis-à-vis des détails d'implémentation. La Clean Architecture résout ces deux problèmes en imposant une règle de dépendance stricte : les couches internes ne connaissent pas les couches externes.

5.1 2.1 ORGANISATION EN COUCHES

Le projet est organisé en cinq couches distinctes.

Couche	Dossier	Rôle
shared	src/shared/	Composants transverses (patron Observer)
Domain	src/domain/	Entités métier, interfaces, énumérations
Factories	src/factories/	Création des objets du domaine (patron Factory)
Application	src/application/	Services - orchestration des cas d'usage
Infrastructure	src/infrastructure/	Persistance SQLite - implémentation des repositories
Présentation	src/presentation/	Interface graphique PySide6

5.2 2.2 REGLE DE DEPENDANCE

La règle fondamentale est que les dépendances ne peuvent pointer que vers l'intérieur. La couche Domaine ne connaît rien des autres couches. Les services de la couche Application dépendent des interfaces définies dans le Domaine, pas des implémentations concrètes.

Concrètement, un service comme EventService dépend de IEventRepository (une interface), et non de SQLiteEventRepository (l'implémentation concrète). Si l'on décide un jour de passer à PostgreSQL ou à un stockage en mémoire pour les tests, il suffit de créer une nouvelle implémentation de l'interface sans toucher à un seul service.

5.3 2.3 PRINCIPES SOLID APPLIQUES

Tout au long du développement, les principes SOLID ont guidé les décisions de conception.

Le principe de responsabilité unique (SRP) est respecté : chaque classe a une seule raison de changer. Les factories s'occupent uniquement de la création, les services uniquement de l'orchestration, les repositories uniquement de la persistance.

Le principe ouvert/fermé (OCP) est appliqué notamment sur les modules : ajouter un nouveau module ne nécessite pas de modifier le code existant, juste d'étendre le système par de nouvelles classes.

Le principe d'inversion de dépendance (DIP) est la colonne vertébrale de l'architecture : tous les services dépendent d'abstractions (interfaces), jamais d'implémentations concrètes.

6 3. DIAGRAMMES UML

6.1 3.1 DIAGRAMME DE CLASSES

Le diagramme de classes est l'outil central de la conception orientée objet. Il permet de représenter la structure statique d'un système : les classes, leurs attributs, leurs méthodes, et les relations qui les unissent. C'est le document de référence qui permet de comprendre comment le système est organisé sans avoir à lire le code.

Le diagramme ci-dessous représente l'ensemble du modèle objet du projet INLINE. Il couvre les six couches de l'architecture : shared, domain, factories, application, et infrastructure.

Plusieurs types de relations sont représentés.
L'héritage relie Observable à tous les services (IntentionService, EventService, NoteService, ModuleService, EnvironmentService), ce qui matérialise le patron Observer dans le diagramme.
L'implémentation d'interface est utilisée pour les blocs de notes (TitleBlock, TextBlock, ChecklistBlock, TableBlock implémentent INoteBlock) et pour les repositories SQLite (chaque implémentation concrète implémente son interface correspondante dans le domaine), ce qui matérialise le patron Factory et le principe DIP.
La composition relie Environment à IInteractiveObject et Note à INoteBlock. Ces objets ne peuvent pas exister indépendamment de leur conteneur.
L'agrégation relie PomodoroSession à PomodoroModule. Une session référence un module mais peut exister indépendamment.
Les associations avec multiplicités indiquent par exemple qu'un utilisateur peut posséder plusieurs environnements et plusieurs intentions, qu'un événement est lié à exactement une intention et un environnement, et qu'une note peut être liée à zéro ou une intention.

On identifie le rôle de chaque classe : «interface» pour les contrats abstraits, «Factory» pour les factories, «Observer» pour les services et la classe Observable, et «enumeration» pour les enums du domaine.

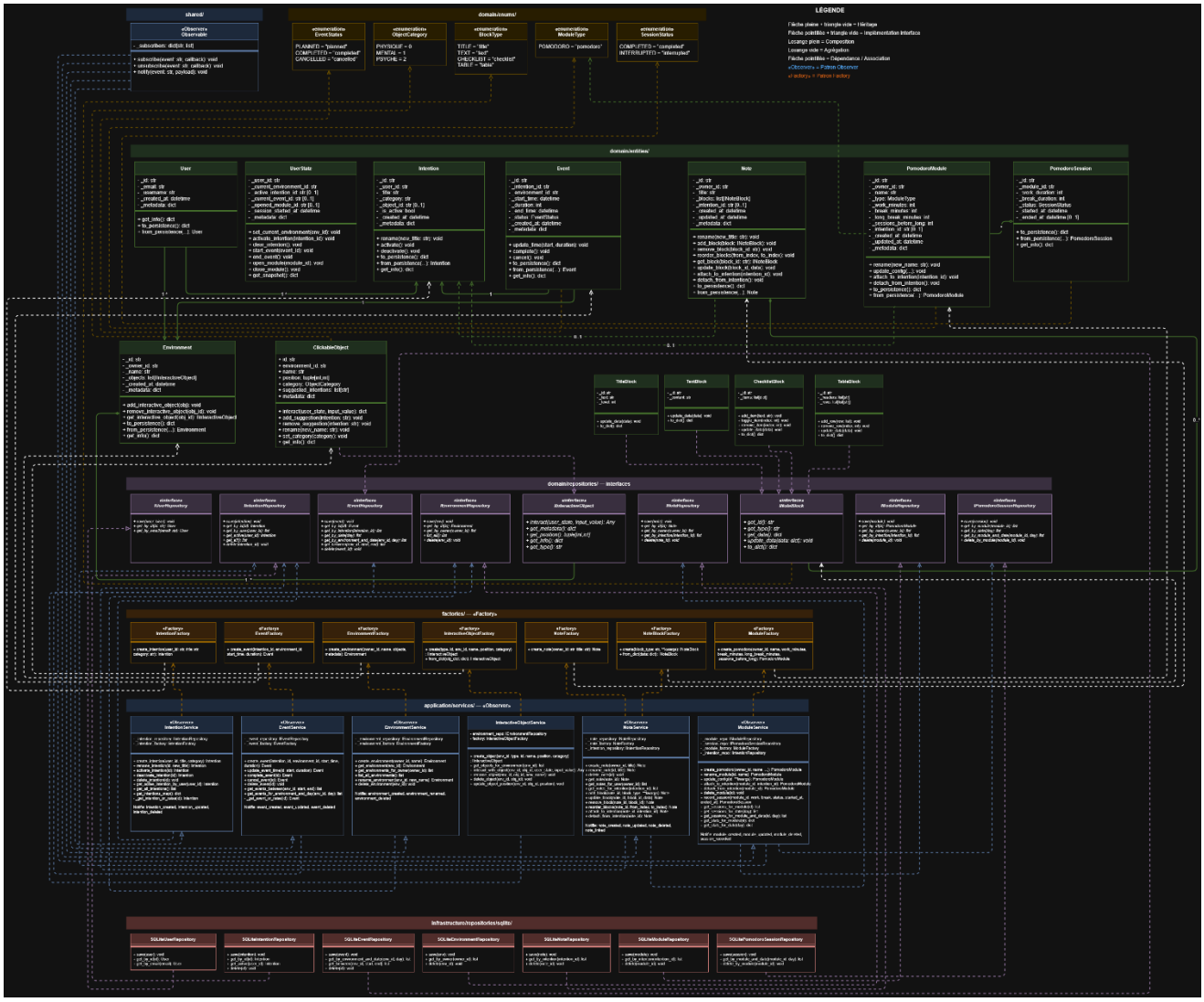

Figure 6 — Diagramme de classes UML complet du projet INLINE (disponible dans INLINE/docs)

6.2 3.2 DIAGRAMME DE SEQUENCE

Le diagramme de séquence est un outil UML comportemental qui représente les interactions entre les objets d'un système dans le temps. Contrairement au diagramme de classes qui montre la structure statique, le diagramme de séquence montre ce qui se passe concrètement lors de l'exécution d'un processus : qui appelle qui, dans quel ordre, et avec quels paramètres. C'est le document qui permet de comprendre comment les couches de l'architecture collaborent pour réaliser une fonctionnalité.

Le processus choisi est la création d'une note. Ce cas a été retenu car il est représentatif du fonctionnement général du système et met en évidence simultanément les deux patrons de conception du projet : Factory pour la construction de l'objet, et Observer pour la propagation du changement vers l'interface.

La séquence se déroule en sept étapes. L'utilisateur clique sur le bouton de création et saisit un titre via une boîte de dialogue Qt. NotesPage transmet la demande à NoteService en appelant

create_note(owner_id, title). NoteService délègue immédiatement la construction de l'objet à NoteFactory, qui génère un UUID, initialise les valeurs par défaut (liste de blocs vide, timestamps, métadonnées) et instancie l'objet Note. NoteService reçoit l'objet construit et le persiste via INoteRepository, qui exécute un INSERT en base SQLite. Une fois la persistance confirmée, NoteService appelle notify('note_created', note) sur Observable, qui itère sur tous les abonnés enregistrés pour cet événement. NotesPage, qui s'est abonnée au démarrage, reçoit le callback et recharge automatiquement sa liste de notes.

On peut remarquer l'absence de couplage direct entre NoteService et NotesPage. Le service ne connaît pas l'interface, et l'interface n'a pas besoin d'interroger le service. C'est le mécanisme Observer qui assure la synchronisation entre les deux couches.

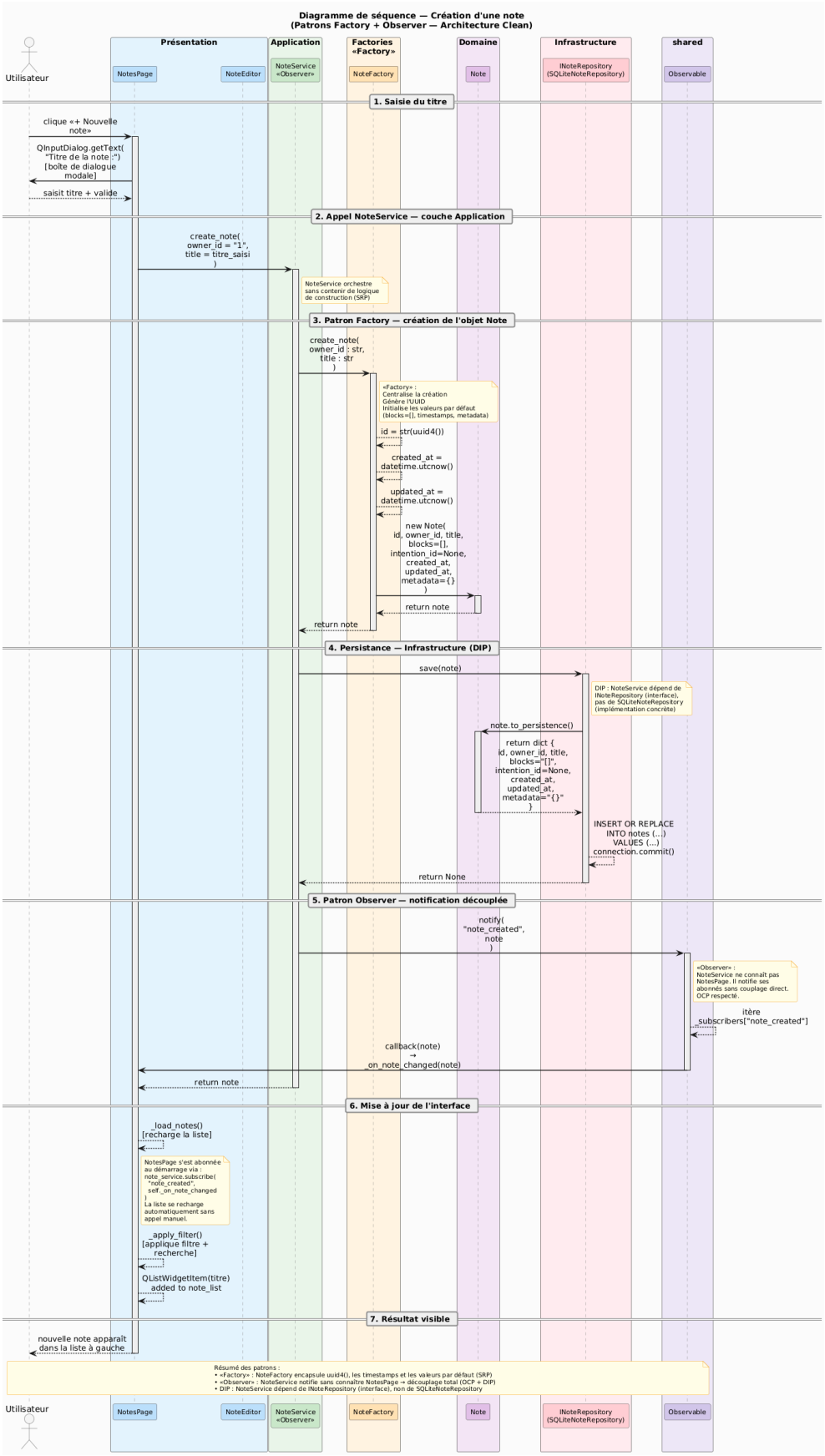

Figure 7 — Diagramme de séquence — Création d'une note (disponible dans INLINE/docs)

7 4. PATRONS DE CONCEPTION

7.1 4.1 PATRON OBSERVER (COMPORTEMENT)

Le patron Observer est un patron de conception comportemental qui définit une relation de dépendance entre objets : quand un objet change d'état, tous ses dépendants sont notifiés et mis à jour automatiquement. Il repose sur deux rôles distincts : l'observable, qui émet des notifications, et les observateurs, qui s'y abonnent et réagissent.

Dans ce projet, le patron a été choisi pour résoudre un problème concret : comment mettre à jour l'interface graphique automatiquement quand le domaine change, sans créer de couplage direct entre les services et les composants visuels ? Sans ce patron, chaque service aurait dû contenir des références directes aux pages de l'interface pour les mettre à jour, ce qui aurait violé le principe d'inversion de dépendance et rendu le système difficile à faire évoluer.

La classe Observable est définie dans le dossier shared/ et héritée par tous les services : IntentionService, EventService, NoteService, ModuleService et EnvironmentService. Elle expose trois méthodes : subscribe() pour s'abonner à un événement nommé, unsubscribe() pour se désabonner, et notify() pour déclencher tous les callbacks enregistrés pour un événement donné.

En pratique, quand un utilisateur crée un événement, EventService appelle notify('event_created', event) après avoir persisté les données. VisualizationPage, qui s'est abonnée à cet événement au démarrage via event_service.subscribe('event_created', self._on_event_changed), reçoit la notification et déclenche automatiquement le rechargement de la timeline. Le service ne sait pas que VisualizationPage existe, et VisualizationPage n'a pas besoin d'interroger périodiquement le service pour savoir si quelque chose a changé.

Ce découplage a un avantage direct sur l'extensibilité. Si demain on ajoute un tableau de bord analytique qui doit aussi réagir à la création d'un événement, il suffit de l'abonner à l'événement correspondant. Aucun service n'a besoin d'être modifié.

Événement notifié	Déclenché par
intention_created / updated / deleted	IntentionService
event_created / updated / deleted	EventService
environment_created / renamed / deleted	EnvironmentService
note_created / updated / deleted / linked	NoteService
module_created / updated / deleted / session_recorded	ModuleService

7.2 4.2 PATRON FACTORY (CREATION)

Le patron Factory est un patron de conception de création qui délègue la responsabilité de l'instanciation des objets à une classe dédiée. Plutôt que d'appeler directement le constructeur d'une classe depuis n'importe quel endroit du code, on passe par une factory qui centralise cette logique et la cache derrière une interface simple.

Dans ce projet, le patron a été choisi pour plusieurs raisons. La création des objets du domaine n'est pas triviale. Elle implique de générer un identifiant unique, d'initialiser des valeurs par défaut cohérentes, de calculer certains champs dérivés, et parfois de valider les paramètres avant construction. Si cette logique était dispersée dans les services, chaque service deviendrait responsable de la construction en plus de l'orchestration, ce qui violerait le principe de responsabilité unique. De plus, centraliser la création facilite l'écriture des tests : pour tester une entité, il suffit d'appeler la factory avec les paramètres voulus sans avoir à reconstruire manuellement l'objet à chaque fois.

Chaque entité principale dispose de sa propre factory : IntentionFactory, EventFactory, NoteFactory, NoteBlockFactory, EnvironmentFactory, InteractiveObjectFactory et ModuleFactory. Chaque factory encapsule la génération de l'identifiant unique via uuid4(), l'initialisation des timestamps (created_at, updated_at), l'initialisation des collections vides (liste de blocs pour une note, liste d'objets pour un environnement), et les validations à la construction comme vérifier qu'une durée est strictement positive avant de créer un événement.

EventFactory.create_event() illustre bien ce fonctionnement : elle prend en paramètres l'intention, l'environnement, l'heure de début et la durée, génère l'identifiant, calcule automatiquement l'heure de fin à partir de start_time et duration, initialise le statut à PLANNED, et retourne un objet Event complet et cohérent. EventService n'a pas à connaître ces détails de construction.

NoteBlockFactory est un cas particulier qui s'apparente davantage à une Abstract Factory : elle ne crée pas un seul type d'objet mais choisit dynamiquement entre TitleBlock, TextBlock, ChecklistBlock et TableBlock selon le type passé en paramètre. Elle centralise aussi la désérialisation depuis la base de données via sa méthode from_dict(), qui reconstruit le bon type de bloc à partir d'un dictionnaire JSON chargé depuis SQLite.

8 5. OUTILS DE GENIE LOGICIEL

8.1 5.1 GIT

Git a été utilisé tout au long du projet pour la gestion des versions. Le repository est hébergé sur GitHub à l'adresse suivante : https://github.com/Caapo/INLINE.

Git permet de conserver un historique complet de toutes les modifications apportées au code. Chaque état du projet est sauvegardé sous forme de commit, ce qui permet de revenir à n'importe quelle version antérieure en cas de problème, de comparer deux états du code, ou de comprendre pourquoi une modification a été faite et à quel moment.

Les commits ont été effectués régulièrement, au fil du développement des fonctionnalités. Chaque commit correspond à une unité de travail cohérente : ajout d'une entité du domaine, implémentation d'un service, création des tests unitaires, modification d'un fichier de présentation...

GitHub a également servi de point de sauvegarde centralisé, garantissant que le code n'est pas uniquement stocké en local. C'est aussi via GitHub que l'on peut accéder au code source et consulter l'historique des commits.

8.2 5.2 DOCKER

Docker est un outil de conteneurisation qui permet d'empaqueter une application avec toutes ses dépendances dans un environnement isolé et reproductible. L'intérêt principal est qu'un conteneur Docker se comporte de manière identique quelle que soit la machine sur laquelle il tourne : le code qui fonctionne dans le conteneur fonctionnera de la même façon sur n'importe quel autre système disposant de Docker, sans avoir à installer manuellement Python, les librairies Qt ou les dépendances du projet.

Dans le cadre de ce projet, l'application étant une application desktop avec interface graphique, Docker n'est pas utilisé pour lancer l'interface elle-même, ce qui nécessiterait une configuration d'affichage graphique complexe. Il est utilisé pour exécuter les tests unitaires dans un environnement Linux isolé et reproductible.

Le Dockerfile part d'une image Python 3.12 officielle, installe les librairies système nécessaires à Qt (libgl1, libglib2.0, libdbus, libxcb), puis installe les dépendances Python via requirements.txt. La variable d'environnement QT_QPA_PLATFORM=offscreen indique à PySide6 de s'initialiser en mode sans affichage physique, ce qui permet aux tests unitaires de s'exécuter sans écran réel.

Le docker-compose.yml orchestre le lancement du conteneur. Les dossiers src/ et tests/ sont montés en volumes, ce qui signifie que les fichiers du projet sur la machine hôte sont directement accessibles dans le conteneur. Cela permet de relancer les tests après une modification du code sans avoir à reconstruire l'image, ce qui accélère le cycle de développement.

Commande	Description
docker build -t inline-app .	Construit l'image Docker (première fois)
docker-compose up	Lance les tests dans le conteneur
docker-compose down	Supprime le conteneur
docker-compose up --build	Reconstruit et relance (après ajout de dépendance)

Les 47 tests unitaires passent avec succès dans le conteneur Docker.

9 6. TESTS UNITAIRES

Les tests unitaires couvrent les entités du domaine, qui constituent le noyau du système. Ils ont été écrits avec pytest et sont regroupés dans le fichier tests/test_domain.py.

Le choix de tester uniquement la couche domaine est délibéré. Ce sont ces classes qui contiennent la logique métier du système, donc les règles les plus importantes à protéger. Les services, les repositories et l'interface graphique ne sont pas testés dans le cadre de ce projet, car ils dépendent d'effets de bord (base de données, affichage) qui rendraient les tests plus complexes à mettre en place.

Classe testée	Tests	Exemples de cas couverts
Intention	7	Activation, désactivation, renommage, persistance
Event	9	Statut par défaut, calcul end_time, complétion, annulation, reprogrammation
Note	10	Ajout/suppression/réordonnancement de blocs, lien intention
NoteBlocks (4 types)	9	Lecture/écriture des données, sérialisation
PomodoroModule	8	Configuration, renommage, lien intention, persistance
PomodoroSession	4	Création, statuts, persistance avec ended_at null
Total	**47**	**Pas d'échec**

10 7. ORGANISATION DU TRAVAIL

10.1 7.1 TABLEAU DES TACHES

Tâche	Durée estimée	Durée réelle	Statut
Réflexion projet et conception de l'architecture	x	7 jours	Terminé
Backend – Intentions (entité, factory, service, repo)	1 jour	1 jour	Terminé
Backend – Event (entité, factory, service, repo)	1 jour	1 jour	Terminé
Backend – Environments et objets (entité, factory, service, repo)	1 jour	2 jours	Terminé
Backend – Modules Pomodoro (entité, factory, service, repo)	1 jour	1 jour	Terminé
Frontend - Visualisation	1,5 jour	4 jours	Terminé
Frontend - Notes	1 jour	1 jour	Terminé
Frontend - Modules (Pomodoro)	1 jour	1 jour	Terminé
Tests unitaires	0,5 jour	0,5 jour	Terminé
Docker	0,5 jour	0,5 jour	Terminé
README, diagrammes UML, rapport	1,5 jour	3 jours	Terminé

Figure 8 — Tableau des tâches du projet – (x jours = étendu sur x jours, et non x jours de code pur)

10.2 7.2 DIFFICULTES RENCONTREES

10.2.1 Évolution du modèle conceptuel

Lors de la formalisation de la vision technique à travers le diagramme de classes, plusieurs incohérences sont apparues entre les choix initiaux présentés et une conception réellement optimale. Par exemple, le fait qu'un module puisse créer directement des intentions ou encore la présence d'espaces composés d'environnements introduisaient une complexité et un flou conceptuel inutiles.

Pour corriger cela, une démarche progressive a été adoptée : chaque composant du système a d'abord été défini de manière isolée, avant de traiter les relations de composition. Cette approche a permis de clarifier les responsabilités et d'aboutir à un modèle plus cohérent. Notamment, l'unité principale de la visualisation a été recentrée sur la notion d'environnement plutôt que d'espace, afin de mieux refléter des contextes concrets et manipulables (ex : chambre, cuisine), facilitant ainsi la structuration des actions quotidiennes.

10.2.2 Généralisation des objets interactifs

Initialement, seuls des objets interactifs destinés à être ajoutés à la timeline étaient envisagés. Cette vision s'est révélée trop restrictive, notamment dans une optique d'extension du système, où certains objets pourraient avoir un rôle purement informatif ou d'observation.

La solution a été d'introduire une interface générique pour les objets interactifs, plutôt qu'une implémentation unique. Cela permet d'étendre facilement le système à différents types d'objets, tout en conservant une architecture flexible et évolutive.

10.2.3 Flexibilité des intentions

Une limitation importante identifiée concernait la rigidité des intentions prédéfinies. Proposer une liste fixe réduisait fortement la capacité d'expression de l'utilisateur et allait à l'encontre de l'objectif du projet, centré sur la personnalisation.

Les intentions ont donc été repensées pour accepter des entrées utilisateurs dynamiques. Cette évolution soulève naturellement la question du rôle des objets interactifs. Ceux-ci conservent néanmoins une importance centrale :

- ils ancrent l'intention dans un contexte réel,
- ils fournissent une localisation mentale,
- ils facilitent la visualisation spatiale,
- ils permettent l'émergence de patterns analysables,
- ils donnent du sens aux actions (une même intention pouvant avoir des significations différentes selon le contexte).

10.2.4 Positions des objets interactifs

Au départ, les positions étaient stockées en pixels absolus. Quand la fenêtre était redimensionnée, les objets sortaient des limites. La solution a été de stocker les positions en pourcentage (valeurs entre 0.0 et 1.0) et de les convertir en pixels au moment de l'affichage selon la taille réelle de la zone. Une protection a également été ajoutée pour convertir automatiquement les anciennes valeurs en pixels vers le nouveau format.

10.2.5 Observer et signaux Qt

Qt dispose de son propre système de signaux et slots, qui est distinct du patron Observer implémenté dans shared/. Il a fallu définir clairement quand utiliser l'un ou l'autre : le patron Observer pour les notifications métier entre services et composants UI, et les signaux Qt pour les interactions internes à un widget (drag and drop, clics, etc.). Cette distinction a permis d'éviter un couplage inutile entre les couches.

10.2.6 Sérialisation des blocs de notes

La sérialisation des blocs de notes dans SQLite a posé un défi de conception. Stocker chaque bloc dans une table dédiée aurait complexifié inutilement le schéma. La solution retenue est de sérialiser la liste complète des blocs en JSON dans une seule colonne. La NoteBlockFactory prend en charge la désérialisation lors du chargement depuis la base, en instanciant le bon type de bloc selon la valeur du champ 'type' dans le JSON.

10.2.7 Objets partagés entre plusieurs environnements

Un bug persistant a fait que les objets interactifs créés dans un environnement apparaissaient également dans les autres environnements. Le problème venait de la façon dont les widgets étaient stockés en mémoire. La structure objects_by_env n'était pas correctement initialisée par environnement, ce qui faisait que tous les environnements partageaient la même liste de widgets. La correction a consisté à s'assurer que chaque environnement dispose de sa propre liste indépendante dans le dictionnaire, initialisée à vide à la création de l'environnement, et alimentée uniquement avec les objets chargés depuis la base pour cet environnement précis.

10.2.8 Construction du datetime pour les événements planifiés

Lors de la création d'un événement depuis le dialog, il fallait construire un objet datetime complet à partir de trois sources distinctes : le jour courant affiché dans VisualizationPage, l'heure saisie via QInputDialog sous forme de float (par exemple 8.5 pour 8h30), et les minutes calculées depuis la partie décimale. Les premières versions utilisaient datetime.utcnow().date() pour récupérer le jour, ce qui posait un problème : les événements planifiés sur un jour futur prenaient la date du jour courant. La correction a été de passer explicitement current_day depuis VisualizationPage au dialog, et de construire le datetime à partir de ce jour précis.

11 CONCLUSION

Ce projet a été l'occasion de mettre en pratique de manière concrète les concepts étudiés en cours de Génie Logiciel. La Clean Architecture, les patrons Observer et Factory, les principes SOLID : tout cela existait déjà dans le cours, mais les appliquer sur un projet complet de cette taille donne une compréhension différente et beaucoup plus solide de leur intérêt réel.

Le résultat final est une application fonctionnelle, structurée et documentée. L'architecture choisie a tenu ses promesses : les modifications apportées en cours de développement (changement du format de persistance des positions, ajout de fonctionnalités) ont pu être réalisées sans remettre en cause le noyau du système.

Plusieurs fonctionnalités ont été volontairement laissées de côté pour ne pas dépasser le cadre du projet : l'état analytique (quatrième pilier), et la clôture quotidienne (permettant d'avoir un résumé de la journée sur ce que l'on a réalisé). Ces éléments seront implémentés dans une version future sans modifier l'architecture existante.

Ce projet m'a aussi confronté à des problèmes pratiques qui ne se posent pas dans les exercices de cours : la gestion des positions relatives dans une interface redimensionnable ou encore la sérialisation d'une structure de données composite dans une base relationnelle. Ces difficultés ont eu des solutions, mais elles ont surtout permis de comprendre pourquoi les patterns existent : pas pour suivre des règles, mais pour résoudre des problèmes réels.

Au cours de la licence, j'ai eu l'occasion de réaliser plusieurs projets, au sein desquels la dette technique s'est révélée être un problème récurrent. Ce projet, appuyé par les enseignements du cours de Génie Logiciel, m'a permis de mieux l'anticiper et de la réduire de manière significative.

J'ai également pris conscience des limites de la sur-conception. Chercher à définir une architecture parfaitement aboutie en amont s'est avéré contre-productif, ralentissant fortement le démarrage du développement. En pratique, certains aspects ne deviennent visibles qu'au moment de l'implémentation. Il est donc inévitable que des éléments n'aient pas été anticipés.

Cependant, une réflexion structurée, guidée par les principes du génie logiciel, permet de limiter ces imprévus et d'y répondre de manière maîtrisée. L'enjeu n'est pas d'atteindre une conception parfaite dès le départ, mais de construire une base suffisamment solide pour évoluer de manière cohérente tout au long du développement.